CS3460: Data Structures

Lab 05: Priority Queues

Total Points: 100

Problems

Notes on Grading: Grading will be handled via Gradescope. Any file you submit must begin with the following package statement to be accepted:

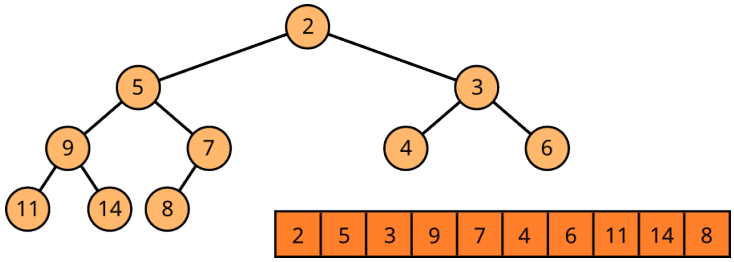
```
1 package student;
```

Your code will be compiled and run against a series of tests. You may submit as many times as you would like before the assignment closes. To earn full credit, you must submit to both Gradescope and AsULearn before the assignment deadline.

Important Note: You are not allowed to use any classes or code from the Java Collections library (built-in Java data structures). The purpose of these assignments is to build these data structures from first principles. Programs which use any of these libraries will receive zero points. This only applies to Collections, and does not include classes like `Scanner` or other utility classes.

Submission: Please submit the files `BinaryHeap.java` and `MedianFinder.java`.

1. **Building a Binary Heap (60 points):** A *binary heap* is an almost-complete binary tree that satisfies the heap property, which is that the value stored at every node greater than or equal to the value stored in its parent, or key(parent(e)) $\leq$ key(e). All levels are filled to capacity before moving to the next level, and each level is filled from left to right. While this isn't necessary for the heap property to be maintained, this added constraint allows us to embed our tree into an array.



An array embedding of a binary heap.

You will implement a binary min-heap (meaning lower key = higher priority) in a file called `BinaryHeap.java`. This class should provide the following operations:

(a) `public void insert(int key)` : insert an integer `key`.

(b) `public int remove_min()` : remove the minimum key and return it.

(c) `public int find_min()` : return the minimum key (without removing).

(d) `public int size()` : return how many elements are in the priority queue

To implement these methods, you will want to create methods for `sift_up(i)` and `sift_down(i)` to maintain the heap property. With these, we can implement *insert* and *remove-min* by

(a) `public void insert(int key)` : placing element at the end of the array (may need to resize the array), then `sift_up(n)` .

(b) `public int remove_min()` : swapping the minimum (at $i = 0$) with the last element of the array, then `sift_down(0)` .

Create a driver to test these operations thoroughly for correctness and efficiency.

2. **Online Median Finding (40 points):** The *median* of a collection of values is the middle value in a sorted output. This simple definition for the median gives a straightforward way to find it: sort the list and look at the value at index $n/2$. This takes $O(n \lg n)$. There are other algorithms for finding the median, such as *quickselect*, but sorting is easy!

These solutions, however, assume that you have the entire list. What if you knew you needed to be able to find the median, while new values were still arriving? How would you do that?

Incidentally, another way to define the median of a dataset is as the value m for which 50% of the values are less than m and 50% of the values are greater than m. This suggests another possible solution for finding the median: What if we maintained two priority queues, one with the **large** half of the dataset, and one with the **small** half of the dataset? If these priority queues were balanced, the median would either be the max of **small** or the min of **large**.

Unfortunately, we implemented a min-heap, which won't give us what we are looking for in **small**. Really, the **small** values should be using a max-heap. How can we use a min-heap as a max-heap? Store the inverse (negative) of every value.

Therefore, the strategy looks like this:

(a) When a new element arrives, choose whether to insert into **large** or **small** by calling *find-min* on **large** to decide.

(b) If the priority queues become imbalanced as a result (their sizes differ by 2), move an element from the one with more elements to the one with fewer elements (keep in mind that one is storing negative values and one is storing positive values). This will restore the balance.

(c) When asked for the median, *find-min* on the priority queue with more elements. If they are the same size, *find-min* on both of them and compute the arithmetic mean on the two values.

In this problem, the input is considered to be "online", meaning you will be answering queries about the data before all of it has arrived. You will write a file named `MedianFinder.java` with two methods:

(a) `public void insert(int v)` : insert a value into the dataset `key`.

(b) `public int getMedian()` : return the median of the values in the dataset. if there are an even number of values, return the arithmetic mean of the two median values

A `main()` method has been provided that will parse an input file for local testing.